



Android Push Notifications with GCM

This article explores the use of the Google Cloud Messaging service to send push notifications to applications running on Android phones.

Mobile applications are all about delivering instant information on the move, but who would like to have an application where you keep pressing the *Refresh* button to fetch the latest updates? An auto-refresh after every minute to check for updates sounds like a solution, but that would involve some major challenges. A mobile device has limited resources in terms of battery life and data transfer limits—not to mention the server load that millions of devices would generate by contacting it every minute.

Wouldn't it be great if the server could automatically tell all the devices that there is new information available for them, as soon as the data arrives? Here, push notifications come to the rescue! Most major mobile OSs offer pretty similar push notification functionality to developers. Apple Push Notifications Service (APNS) for the iPhone, Google Cloud Messaging (previously C2DM) for Android, and Microsoft Push Notifications (MPNS) are the popular services.

A typical push notification system follows the journey depicted in Figure 1. On mobile app installation, the device connects to the push notifications' servers (owned by Google, Apple or Microsoft) and requests for a unique device ID. This is then sent to your application server (see Figure 2) and is used for sending notifications to this device. To send a message, your application server sends the notification content (along with the device ID to which the message is to be sent) to the push server, which takes the responsibility of sending this message to the device, and returns a success/error response to the application server.

The Google Cloud Messaging service was launched at Google's annual I/O event in June 2012, and it replaced C2DM (cloud to device messaging), which was the beta version. With GCM, the limitation on the number of messages per day was removed, and a multi-cast feature was introduced.

It is possible to implement the server side and client side of GCM by making individual service requests, but Google has made things simpler by providing two easy-to-use libraries. To get started, download the GCM library using the Android SDK, and get the jar files from 'ANDROID_HOME\extras\google\gcm'.

First, you need to register an application with Google. Go to <https://code.google.com/apis/console> and create a new project. Navigate to the *Services* tab and enable the Google Cloud Messaging service for this project. Navigate to the *API Access* tab and create a new server key, which would be used on the server end to send messages. Also note down the project ID from the URL; this would be used in the Android application to get the messages.

Next, you need to develop an Android application.



Note: You can download the sample server side code and Android application from <http://goo.gl/9SDB4>.

Create a new Android application, and add *gcm.jar* from the Android SDK to the *CLASSPATH*. Modify the manifest file to have the following:

- Permissions for Internet, the ability to access accounts and receive GCM messages, as well as a wake lock
- A service
- A broadcast receiver
- A custom permission to prevent other applications from receiving the notifications.
- SDK version 8+

Given below is the manifest for my application:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="in.amolgupta.android.gcm"
    android:versionCode="1"
    android:versionName="1.0" >
    <uses-sdk android:minSdkVersion="8" />
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.GET_ACCOUNTS" />
</>
    <uses-permission android:name="android.permission.WAKE_LOCK" />
    <permission
        android:name="in.amolgupta.android.gcm.permission.C2D_
MESSAGE"
        android:protectionLevel="signature" />
    <uses-permission android:name="com.google.android.c2dm.
permission.RECEIVE" />
    <uses-permission android:name="in.amolgupta.android.gcm.
permission.C2D_MESSAGE" />
    <uses-permission android:name="android.permission.GET_ACCOUNTS" />
</>
    <uses-permission android:name="android.permission.USE_
CREDENTIALS" />
    <uses-permission android:name="android.permission.WRITE_
EXTERNAL_STORAGE" />
    <application
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name" >
        <activity
            android:name=".SamplePushActivity">
```